

QHSE Analytics

Architecture * Backend Spring Boot * Frontend Angular

Zeineb Ben Jeddou - 2026

Table des matieres

1.	Vue d'ensemble du projet
2.	Architecture globale et stack technique
3.	Base de donnees - Schema et migrations Flyway
4.	Securite - JWT, OTP, Rate Limiting
5.	Pipeline d'import Excel (8 etapes)
6.	Calcul KPI et Classification
7.	Integration LLM - Groq
8.	Chaine LLM avec Fallback (LlmProviderChain)
9.	Google Gemini - Fallback et Embeddings
10.	Systeme RAG - pgvector et Recherche Vectorielle
11.	Prompt Engineering et Reponse Structuree
12.	Frontend Angular - Architecture et Signals
13.	Composant Analyse IA (654 lignes)
14.	Cache, Performance et Metriques
15.	Deploiement Docker
16.	Questions-Reponses Soutenance

1. Vue d'ensemble du projet

Objectif

QHSE Analytics est une plateforme SaaS d'analyse des indicateurs QHSE (Qualite, Hygiene, Securite, Environnement) pour les entreprises industrielles. Elle automatise l'analyse des donnees de performance operationnelle en combinant un pipeline de traitement de fichiers Excel, des algorithmes de classification, et des modeles de langage (LLMs) enrichis par un systeme RAG.

Problemes resolus

- Import manuel fastidieux de fichiers Excel → automatisation complete du pipeline
- Calcul manuel des variations N vs N-1 → calcul automatise, classe et visualise
- Absence d'analyse intelligente des KPIs → insights IA avec causes racines et plans d'action
- LLM sans contexte metier → RAG injecte les definitions et seuils QHSE officiels
- Gestion multi-utilisateurs avec roles → admin vs analyste avec authentication securisee

Fonctionnalites principales

- Upload et parsing automatique de fichiers Excel multi-periodes
- Detection automatique des colonnes et fuzzy matching des KPIs (Levenshtein, seuil 0.85)
- Calcul des variations (%), tendances (HAUSSE/BAISSE/STABLE) et niveaux (FAIBLE/MODERE/CRITIQUE)
- Analyse IA structuree : insights, causes racines, alertes predictives, plans d'action
- Base de connaissances RAG vectorielle (pgvector + embeddings Gemini 768D)
- Tableaux de bord interactifs, export PDF (iText7) et Excel (Apache POI)
- Gestion des utilisateurs, audit log, configuration IA a chaud (sans redemarrage)

Acteurs du systeme

- Analyste QHSE : importe les fichiers Excel, visualise les KPIs, consulte les analyses IA
- Administrateur : gere les utilisateurs, le catalogue KPI, la base RAG, la config IA
- Systeme IA : Groq (LLM primaire) + Gemini (fallback LLM + embeddings 768D)

2. Architecture globale et stack technique

Schema d'architecture

```
Angular SPA (Nginx en prod)
  | /api (proxy dev :4200→:8080, Nginx en prod)
Spring Boot REST API (:8080)
  |
PostgreSQL 15 (pgvector extension)
  |      |
Groq API  Google Gemini API
(LLM primaire) (fallback LLM + embeddings 768D)
```

Stack Backend

- Spring Boot 3.3.5 (Java 17) - framework principal REST API
- Spring Security + JWT cookie HttpOnly - authentication et autorisation
- Spring Data JPA / Hibernate - acces base de donnees ORM
- PostgreSQL 15 avec extension pgvector - stockage relationnel + vectoriel
- Flyway - versionning et migration du schema DB (V1-V7)
- Caffeine Cache - cache en memoire (analyses IA 24h TTL, embeddings)
- Apache POI - generation Excel ; iText7 - generation PDF
- Micrometer + Prometheus - metriques IA (cache hits, provider selectionne)
- JavaMail - emails (Gmail SMTP) pour OTP et verification de compte

Stack Frontend

- Angular 21 - Standalone Components (sans NgModules)
- Angular Signals - etat reactif (signal(), computed(), effect())
- Angular Material - composants UI (tables, dialogs, tabs, chips)
- RxJS - gestion des flux asynchrones HTTP
- TypeScript strict mode
- Nginx - serveur web + proxy inverse en production

Stack IA / LLM / RAG

- Groq API - LLM primaire : llama-3.3-70b-versatile, meta-llama-4-scout, qwen3-32b
- Google Gemini API - gemini-2.0-flash (fallback) + text-embedding-2 (vecteurs 768D)
- pgvector - stockage et recherche vectorielle cosinus dans PostgreSQL
- Index IVFFlat - Approximate Nearest Neighbor pour performances vectorielles

Packages backend

com/QHSEAnalytics/
auth/ JWT, OTP, email verification, refresh tokens, audit log
analytics/ Orchestration IA, Groq, Gemini, RAG, dashboards (27 classes)
kpi/ Catalogue KPI CRUD et enrichissement
importer/ Upload → nettoyage → classification → calcul (24 classes)
export/ Generation PDF (iText7) et Excel (Apache POI)
shared/ Entites JPA, repositories, DTOs, enums, exceptions (120 classes)
security/ JwtAuthFilter, rate-limiting filter
config/ Security, CORS, async, Caffeine, OpenAPI, pgvector init

3. Base de donnees - Schema et migrations Flyway

Pourquoi Flyway ?

Flyway garantit que le schema de base de donnees est toujours synchronise avec le code. Chaque migration est un script SQL versionne et immuable. Au demarrage de Spring Boot, Flyway verifie la table 'flyway_schema_history' et applique uniquement les migrations non encore executees. Regle absolue : ne JAMAIS modifier un script existant - creer un nouveau Vn+1.

Migrations V1 a V7

- V1 - Index de performance initiaux (import_sessions, resultat_kpi, users)
- V2 - Colonnes stockage fichier (chemin, nom original, taille) pour les imports
- V3 - Extension pgvector + colonne embedding vector(768) + index IVFFlat sur rag_knowledge
- V4 - Agrandissement colonne code categorie_kpi (contrainte VARCHAR trop courte)
- V5 - Table admin_audit_log (id, user_id, action, timestamp, details JSON)
- V6 - Champ confiance sur analyse_globale + table ai_config (cle/valeur runtime)
- V7 - Champ direction sur rag_knowledge (LOWER_IS_BETTER / HIGHER_IS_BETTER)

Tables principales

>> users

id, email, password (bcrypt), role (ANALYSTE/ADMIN), enabled, emailVerified, createdAt

>> import_sessions

id, user_id (FK), periode, fichierNom, statut (INITIAL/VALIDATED/TRAITE/ERREUR), mode (NORMAL/PREVIEW), createdAt

>> kpi + categorie_kpi

kpi : id, nom, unite, seuilBas, seuilHaut, direction, categorieId (FK). categorie_kpi : id, code (Q/H/S/E), libelle

>> resultat_kpi (table centrale)

id, importSessionId (FK), kpId (FK), valeurN1, valeurN, variationPct, tendance, niveauVariation, analysela (texte legacy), statutNettoyage, createdAt

>> analyse_globale + analyse_categorie

analyse_globale : id, importSessionId, synthese, planActions (JSON), overallConfidence (float).
analyse_categorie : id, importSessionId, categorieCode, contenu (texte IA par categorie)

>> rag_knowledge (table RAG - critique)

id, kpiName, definition, thresholds, category, direction, embedding vector(768). La colonne embedding est un tableau de 768 floats genere par Gemini text-embedding-2. L'index IVFFlat (lists=100) accelere la recherche par similarite cosinus.

```
-- Migration V3
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE rag_knowledge ADD COLUMN embedding vector(768);
CREATE INDEX ON rag_knowledge
  USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);
```

>> ai_config

Table cle/valeur pour la configuration IA runtime (modifiable sans redemarrage). Exemples :
groq.temperature.json=0.1, rag.threshold=0.72, cache.ttl.hours=24

4. Securite - JWT, OTP, Rate Limiting

JWT cookie HttpOnly

Le JWT est stocke dans un cookie HttpOnly (inaccessible depuis JavaScript), ce qui elimine les attaques XSS ciblant le token. Contrairement au localStorage, le cookie est envoye automatiquement par le navigateur a chaque requete vers le meme domaine.

- Access token : TTL 30 minutes
- Refresh token : TTL 1 heure (7 jours avec remember-me), stocke en base
- CookieTokenService : gere la creation, le rafraichissement et la suppression des cookies
- JwtAuthFilter : filtre Spring Security qui valide le JWT a chaque requete
- JwtService : genere et valide les tokens (cle secrete $\geq$ 256 bits dans .env)

Flux d'authentification complet

1. POST /api/auth/login → verification email+password (bcrypt)
2. Si OTP active → envoi code 6 chiffres par Gmail SMTP (TTL 5 min)
3. POST /api/auth/verify-otp → validation code
4. Emission JWT access + refresh tokens en cookies HttpOnly
5. Frontend relaie les cookies automatiquement (withCredentials: true)
6. Token expire → ErrorInterceptor appelle POST /api/auth/refresh
7. Refresh expire ou invalide → redirection login

Rate Limiting - protection brute force

RateLimitingFilter : filtre Servlet sur /api/auth/**. Limite : 5 tentatives par adresse IP sur 300 secondes. Depassement → HTTP 429 (Too Many Requests). Implementation : HashMap<IP, (compteur, timestamp)> en memoire.

Autorisation par roles

- ANALYSTE : acces /api/import/**, /api/ia/**, /api/dashboard/**
- ADMIN : acces /api/admin/**, /api/rag/**, /api/ia-config/**
- Annotations @PreAuthorize sur les controleurs Spring
- Guards Angular (auth.guard, admin.guard, analyste.guard) cote frontend

5. Pipeline d'import Excel (8 étapes)

Le pipeline est orchestre par `KpiProcessingOrchestratorService`. Il transforme un fichier Excel brut en résultats KPI calculés, nettoyés et classés.

Etape 1 - Upload et creation de session

`ImportSessionController.upload()` reçoit le fichier multipart. `LocalFileStorageService` sauvegarde dans `./storage/imports/{sessionId}/`. `ImportSession` crée en base : statut=INITIAL, mode=PREVIEW ou NORMAL.

Etape 2 - Parsing Excel

`ExcelFileValidator` vérifie le format (.xlsx/.xls) et la structure minimale. `ExcelParserUtil` (Apache POI) lit toutes les feuilles et cellules, construit une liste de lignes brutes (`Map<String, String>`).

Etape 3 - Detection des en-tetes

`HeaderDetectionUtil` analyse la première ligne : identifie automatiquement les colonnes nom KPI, valeur N, valeur N-1, unite. Utilise des heuristiques basées sur des mots-cles ('valeur', 'n-1', 'objectif', etc.).

Etape 4 - Fuzzy matching KPI

`KpiMatchingUtil` compare le nom de chaque ligne Excel avec le catalogue KPI en base. Algorithme : distance de Levenshtein normalisée ($\text{similarite} = 1 - \text{distance}/\text{maxLen}$). Seuil : 0.85 (85% de similarité requise). Si match → association `kpild`. Sinon → KPI inconnu (ignore ou crée selon config).

Etape 5 - Extraction

`ExtractionAgent` crée les entités `StagingDonnee` (données brutes) : `kpild`, `valeurN`, `valeurN1`, `unite`, `periode`. Associées à la session d'import.

Etape 6 - Nettoyage (CleaningAgent)

Normalisation des nombres (virgule → point, suppression espaces), détection des valeurs aberrantes (IQR : valeurs au-delà $Q3 + 1.5 * IQR$), traitement des valeurs manquantes. StatutNettoyage : PROPRE, CORRIGE, SUSPECT, INCOMPLET.

Etape 7 - Calcul (CalculationAgent + ComparativeCalculator)

$$\text{variationPct} = ((\text{valeurN} - \text{valeurN1}) / |\text{valeurN1}|) * 100$$

tendance :

si $|\text{variationPct}| < 5\%$ → STABLE

si $\text{variationPct} > 0$ → HAUSSE

sinon → BAISSE

direction LOWER_IS_BETTER si le nom KPI contient :

'incident', 'accident', 'defaut', 'erreur', 'retard', 'absenteisme'

Etape 8 - Classification et enrichissement (ClassificationEngine)

Si direction = LOWER_IS_BETTER :

HAUSSE forte → CRITIQUE (ex: +20% accidents = tres mauvais)

HAUSSE moderee → MODERE

BAISSE/STABLE → FAIBLE

Si direction = HIGHER_IS_BETTER :

BAISSE forte → CRITIQUE (ex: -20% conformite = tres mauvais)

BAISSE moderee → MODERE

HAUSSE/STABLE → FAIBLE

Ensuite : MetadataEnrichmentAgent, RiskDetectionAgent (anomalies combinées), VisualizationAgent (donnees graphiques). ResultatKpi persistes en base. Session passe en statut TRAITE.

Mode PREVIEW vs NORMAL

PREVIEW : calculs stockes temporairement pour validation utilisateur. Validation → conversion NORMAL (statut VALIDATED puis TRAITE). Abandon → suppression automatique apres 15 jours (ImportDataRetentionService).

6. Calcul KPI et Classification

Formules et exemple complet

Variation (%) = $((\text{valeurN} - \text{valeurN1}) / |\text{valeurN1}|) \times 100$

Exemple concret :

KPI : Taux d'accidents

valeurN1 = 5 | valeurN = 6

Variation = $((6 - 5) / 5) \times 100 = +20\%$

Direction : LOWER_IS_BETTER (mot 'accident' dans le nom)

Tendance : HAUSSE

Niveau : CRITIQUE (hausse sur indicateur a minimiser)

Cas division par zero : si valeurN1 = 0 → variationPct = null

→ KPI exclu de l'analyse IA (filtre dans AnalysisAgent)

Rapport qualite des donnees

QualityReportBuilder produit : nb lignes totales, nb corrigees, nb suspectes, nb incompletes, taux de completude, distribution FAIBLE/MODERE/CRITIQUE. Affiche a l'utilisateur avant validation PREVIEW → NORMAL.

Point cle soutenance

Classifi
les mots-
: les tests

7. Integration LLM - Groq

Pourquoi Groq ?

- Inference ultra-rapide via LPU (Language Processing Unit) propriétaire
- Latence 5-10x inférieure à OpenAI pour les mêmes modèles llama
- API compatible OpenAI → migration simplifiée si besoin
- Accès à des modèles open-source puissants : llama-3.3-70b, qwen-3-32b
- Tier gratuit généreux pour le développement et les tests

Modèles configurés

- llama-3.3-70b-versatile : modèle principal (70B paramètres, excellent raisonnement)
- meta-llama/llama-4-scout-17b-16e-instruct : modèle léger et rapide (fallback interne)
- qwen/qwen3-32b : modèle Alibaba (diversité de raisonnement)

Rotation de clés API (GroqKeyRotator)

Groq impose des limites de débit (RPM/TPM) par clé API. GroqKeyRotator contient une liste de clés provenant de .env (GROQ_API_KEYS, séparées par virgule). Sélection en round-robin : chaque appel prend la clé suivante dans la liste. Si une clé reçoit 429 → ProviderCooldownManager la met en cooldown et GroqKeyRotator passe à la suivante disponible.

Paramètres d'appel Groq

POST https://api.groq.com/openai/v1/chat/completions

```
model      : llama-3.3-70b-versatile
messages   : [{role:system, content:...}, {role:user, content:prompt}]
temperature : 0.1 (JSON structure) ou 0.5 (texte libre)
max_tokens  : 2800
response_format: {type: json_object} // garantit JSON valide en sortie
timeout     : 30 secondes
```

Pourquoi temperature 0.1 pour JSON et 0.5 pour texte ?

Temperature = degré de créativité/aleatoire du LLM. 0.0 = déterministe. 1.0 = très créatif. Pour les analyses JSON structurées → 0.1 : précision maximale, respect strict du format, pas de variations inattendues. Pour les synthèses textuelles (plans d'action, recommandations) → 0.5 : formulations plus naturelles et variées.

Méthodes de GroqService

- analyserKpi(kpiName, variation, context) : insight par KPI en JSON
- analyserCategorie(categorie, kpis[]) : synthèse d'une catégorie QHSE

- `genererSynthese(kpisResume)` : synthese globale de tous les KPIs
- `genererPlanActions(kpisAlerte)` : plans d'action pour les KPIs CRITIQUES

8. Chaine LLM avec Fallback (LlmProviderChain)

Architecture de la chaine

```
LlmProviderChain.generate(prompt, cacheKey, bypassCache)
|
|-- 1. Verifier cache Caffeine (cle SHA-256)
|   Si hit → retourner immediatement (0ms LLM)
|
|-- 2. Tenter Groq API (provider primaire)
|   ProviderCooldownManager.isAvailable('groq') ?
|   Oui → appel HTTP, timeout 30s
|   429/timeout → cooldown groq + passer a etape 3
|
|-- 3. Tenter Gemini API (fallback)
|   ProviderCooldownManager.isAvailable('gemini') ?
|   Oui → appel HTTP, timeout 30s
|   429/timeout → cooldown gemini
|
|-- 4. Les deux indisponibles → ProviderUnavailableException
    → AnalysisAgent retourne statut=FAILED + fallbackReason
```

Cache Caffeine dans la chaine LLM

Cle de cache : SHA-256 du cacheKey fourni par AnalysisAgent. Format cle source : importSessionId=X|mode=structured|promptVersion=Y|schemaVersion=Z|chunk=N. TTL : 24 heures (configurable via ai_config). bypassCache=true lors d'une regeneration explicite par l'utilisateur.

ProviderCooldownManager

Gere les periodes de refroidissement apres un 429. Stocke par provider : {lastErrorTime, cooldownDuration}. Cooldown initial : 60 secondes. Augmente exponentiellement si erreurs repetees. isAvailable() retourne false si (now - lastErrorTime) < cooldownDuration.

Metriques Micrometer

- ai.cache.hit : nb de reponses servies depuis le cache
- ai.cache.miss : nb d'appels LLM reels effectues
- ai.provider.selected (tag: groq/gemini) : quel provider a repondu
- ai.retry.count : nb de tentatives de retry JSON
- ai.parse.error.count : JSON invalides recus des LLMs

9. Google Gemini - Fallback et Embeddings

Deux roles distincts de Gemini

- Role 1 - LLM fallback : generation de texte quand Groq est indisponible (gemini-2.0-flash)
- Role 2 - Embeddings RAG : vecteurs 768D pour la recherche semantique (text-embedding-2)

GeminiClientService - LLM fallback

POST <https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent>

Corps :

contents: [{role:user, parts:[{text: prompt}]]}

generationConfig:

responseMimeType: application/json

maxOutputTokens: 2800

Retry : max 5, backoff exponentiel (1s, 2s, 4s, 8s, 60s max)

Gestion 429 : cooldown dynamique via ProviderCooldownManager

EmbeddingService - vecteurs 768D

POST <https://generativelanguage.googleapis.com/v1beta/models/text-embedding-002:embedContent>

Corps : {content: {parts: [{text: texte}]}}

Retourne : {embedding: {values: [768 floats]}}

Troncature : 6000 caracteres max avant envoi

Cache Caffeine : une entree par texte (evite appels API redondants)

Gestion 429 : cooldown dynamique, retourne null si indisponible

Qu'est-ce qu'un embedding 768D ?

Un embedding est une representation numerique d'un texte sous forme de vecteur. text-embedding-2 de Gemini projette n'importe quel texte dans un espace a 768 dimensions. Textes semantiquement similaires → vecteurs proches dans cet espace. Exemple : 'taux d'accidents du travail' et 'frequence des blessures' auront des vecteurs tres proches meme sans mot en commun.

10. Systeme RAG - pgvector et Recherche Vectorielle

Pourquoi le RAG ?

Probleme : un LLM generaliste ne connait pas les definitions precises des KPIs QHSE de l'entreprise, ni ses seuils acceptables, ni la direction d'amelioration. Sans contexte → l'IA produit des insights generiques et potentiellement incorrects.

Solution RAG (Retrieval-Augmented Generation) : avant chaque appel LLM, on recupere depuis la base de connaissances la definition exacte du KPI analyse, ses seuils et sa direction. Ces informations sont injectees dans le prompt. L'IA dispose d'un contexte metier precis et produit des insights pertinents.

Pipeline RAG - Phase 1 : Indexation (admin)

1. Admin cree une entree via RagAdminController (nom KPI, definition, seuils, categorie, direction)
2. RagAdminService appelle EmbeddingService.embed(definition)
3. Gemini text-embedding-2 retourne un vecteur float[768]
4. Le vecteur est stocke dans rag_knowledge.embedding (type vector(768))
5. L'index IVFFlat se met a jour automatiquement

Pipeline RAG - Phase 2 : Recherche lors de l'analyse

1. AnalysisAgent prepare l'analyse d'un KPI (ex: 'Taux d'accidents')
2. Appelle RagKpiKnowledgeLookupService.findRelevant(kpiName, category)
3. EmbeddingService.embed(kpiName) → vecteur de requete float[768]
4. RagSearchService.searchBySimilarity(queryVector, category, topK=3, threshold=0.72)
5. Si resultats insuffisants → searchByKeyword() (ILIKE, threshold=0.50)
6. Contexte RAG injecte dans le prompt LLM

Requete SQL pgvector

```
SELECT id, kpi_name, definition, thresholds, direction,  
       1 - (embedding <=> :queryVector) AS similarity  
FROM rag_knowledge  
WHERE category = :category  
      AND 1 - (embedding <=> :queryVector) >= :threshold  
ORDER BY embedding <=> :queryVector ASC  
LIMIT :topK
```

Operateur <=> : distance cosinus entre vecteurs

1 - distance : similarite cosinus (0.0=opposes, 1.0=identiques)

Similarite cosinus - explication

La similarité cosinus mesure l'angle entre deux vecteurs dans l'espace. Elle est indépendante de la norme (longueur) des vecteurs - uniquement la direction compte. Formule : $\cos(\theta) = (A \cdot B) / (||A|| \times ||B||)$. Valeur 1.0 → vecteurs identiques (textes très similaires). Valeur 0.0 → vecteurs orthogonaux (textes sans rapport). Seuil 0.72 choisi empiriquement pour équilibrer précision et rappel sur les KPIs QHSE.

Index IVFFlat vs HNSW

- IVFFlat : partitionne l'espace en listes (lists=100). Rapide, faible mémoire. Adaptable pour < 100k vecteurs. Utilisez ici car la base RAG QHSE reste petite.
- HNSW : graphe hiérarchique. Plus précis, mais plus gourmand en mémoire. Adaptable pour millions de vecteurs. À envisager si la base RAG dépasse 50k entrées.

Injection du contexte RAG dans le prompt

Contexte métier (base de connaissances QHSE) :

KPI : Taux de fréquence des accidents (TF)

Définition : Nb accidents x 10⁶ / heures travaillées

Seuils : Excellent < 2 | Acceptable 2-5 | Critique > 5

Direction : LOWER_IS_BETTER

Catégorie : Sécurité

Données observées :

Valeur N-1 : 3.2 | Valeur N : 4.8 | Variation : +50%

Tendance : HAUSSE | Niveau : CRITIQUE

Analyse demandée : causes racines, recommandations, alertes prédictives

Fallback keyword si RAG indisponible

Si Gemini Embedding API est indisponible ou similarité < 0.50 : `RagSearchService.searchByKeyword()` utilise ILIKE pour chercher le nom du KPI dans `rag_knowledge.kpi_name`. Moins précis mais toujours fonctionnel.

11. Prompt Engineering et Reponse Structuree

StructuredAnalysisPromptBuilder

Construit le prompt systeme + utilisateur envoye au LLM. Prompt systeme : 'Tu es un expert QHSE senior. Analyse les KPIs fournis et retourne UNIQUEMENT un JSON valide au format specifie.' Prompt utilisateur contient :

- Le schema JSON exact attendu (avec types et descriptions de chaque champ)
- Les donnees des KPIs du batch (nom, variation, niveau, tendance)
- Le contexte RAG enrichi pour chaque KPI (definition, seuils, direction)
- Instructions explicites : 'Ne depasse pas 2800 tokens. JSON uniquement.'

Chunking - pourquoi 5 KPIs par batch ?

Un prompt trop long (100 KPIs) depasse le max_tokens et degrade la qualite. Batch de 5 KPIs : prompt court (~1200 tokens), reponse JSON precise (~800 tokens). AnalysisAgent trie d'abord par |variation| descendant et garde les top 20 pour limiter le nombre de chunks et donc le cout en appels LLM.

StructuredAnalysisValidator - gestion JSON invalide

1. Parse JSON (Jackson ObjectMapper). Si exception → tentative de reparation (extraction substr JSON)
2. Verification champs obligatoires : kpiInsights non vide, chaque insight a kpiName + insight
3. Si invalide → log + retry (max 2 fois). Si echec final → statut=FAILED

Format de reponse structuree (AiAnalysisStructuredResponse)

```
{
  "status": "SUCCESS | PARTIAL | FAILED",
  "confidence": 0.87,
  "kpiInsights": [
    {
      "kpiName": "Taux d'accidents",
      "variation": -12.5,
      "level": "CRITIQUE",
      "trend": "BAISSE",
      "insight": "Description de la situation...",
      "rootCauses": ["Manque EPI", "Formation insuffisante"],
      "recommendations": ["Audit securite", "Formation obligatoire"],
      "ragContext": "Def: ... Seuil: <2% Direction: LOWER_IS_BETTER"
    }
  ],
  "categoryAnalyses": {"S": "...", "Q": "..."},
  "globalSynthesis": "Vision globale...",
  "actionPlans": [{"priority": "HIGH", "action": "...", "deadline": "..."}],
  "predictiveAlerts": [{"alertType": "RISK", "probability": 0.75}]
}
```

12. Frontend Angular - Architecture et Signals

Standalone Components - pourquoi ?

Angular 15+ permet les composants autonomes sans NgModule. Chaque composant declare ses propres imports (Material, RouterModule, etc.). Avantages : tree-shaking plus efficace, code plus lisible, lazy loading facilite. Tous les composants du projet suivent ce pattern.

Angular Signals - etat reactif

```
// Signal : valeur reactive
selectedSessionId = signal<number | null>(null);

// Computed : derive automatiquement
hasData = computed(() => this.kpiInsights().length > 0);

// Effect : reagit aux changements de signal
effect(() => {
  const id = this.selectedSessionId();
  if (id) this.loadAnalysis(id);
});

// ATTENTION : les primitives dans computed() ne triggent PAS
// Faux : filterText = computed(() => "");
// Correct : filterText = signal("");
```

Structure feature analyste

- import/ : upload fichier Excel, suivi de progression
- import-mapping/ : mapping colonnes Excel → KPIs catalogue
- dashboard/ : graphiques KPIs (radar, bar, pie charts)
- historique/ : liste des sessions d'import passees
- comparatif/ : comparaison entre deux periodes
- analyse-ia/ : page principale analyse IA (654 lignes) - cf. ch.13
- export-pdf/ : generation et telechargement PDF

Services principaux

- AiAnalysisService : GET/POST /api/ia/{id}, /api/ia/{id}/structured, /api/ia/{id}/regenerer
- ImportService : POST /api/import/upload, GET /api/import/sessions, POST /api/import/{id}/validate
- DashboardService : GET /api/dashboard/analyste, GET /api/dashboard/admin
- AuthService : login, logout, register, refreshToken, OTP, forgot/reset password

Intercepteurs HTTP

- AuthInterceptor : ajoute withCredentials:true a chaque requete (envoie les cookies JWT)
- ErrorInterceptor : affiche un toast Material sur toute reponse 4xx/5xx

13. Composant Analyse IA (654 lignes)

Logique de chargement des donnees

```
// Effect : charge les donnees a chaque changement de session
effect() => {
  const id = this.sessionId();
  if (id) {
    this.loadStructured(id); // tente le format structure
    this.loadLegacy(id);    // charge aussi le format legacy
  }
};

// Fusion des deux formats
mergedInsights = computed(() => {
  const structured = this.structuredResponse();
  const legacy = this.legacyResponse();
  return this.merge(structured, legacy);
});
```

Fonctionnalites affichees

- Badge de confiance globale (vert ≥ 0.8 , orange 0.5-0.8, rouge < 0.5)
- Liste KPIs avec niveau (FAIBLE/MODERE/CRITIQUE) et variation en %
- Insights par KPI avec contexte RAG enrichi affiche en accordéon
- Causes racines (diagramme Ishikawa textuel)
- Plans d'action (tableau Kanban : HIGH/MEDIUM/LOW priority)
- Alertes predictives (RISK/OPPORTUNITY avec probabilite en %)
- Synthese par categorie (Q, H, S, E) et synthese globale
- Bouton 'Regenerer' qui bypassse le cache (bypassCache=true)

Gestion des etats

- isLoading signal : affiche un spinner Material pendant le chargement
- error signal : affiche un message d'erreur si l'API echoue
- Status SUCCESS : tout s'est bien passe
- Status PARTIAL : analyse incomplete (certains chunks ont echoue) mais affichee quand meme
- Status FAILED : echec total (les deux providers indisponibles)

14. Cache, Performance et Metriques

Cache Caffeine

- kpiAnalysis : analyse complete par session (TTL 24h) - evite reappels LLM couteux
- embeddings : vecteurs Gemini par texte - evite reappels embedding redondants
- CacheConfig.java : chaque cache nomme avec TTL et taille max configures
- Invalidation explicite lors d'une regeneration (evict par sessionId)

@Async - pourquoi ?

L'analyse IA peut prendre 10-60 secondes (plusieurs appels LLM en serie). Si synchrone → le thread HTTP est bloqué → timeout navigateur. @Async('aiAnalysisExecutor') decharge le calcul sur un pool de threads dedie (AsyncConfig : core=4, max=8 threads). Le front fait du polling ou utilise un endpoint de statut pour savoir quand c'est pret.

Optimisations de performance

- Top-20 KPIs : trie par |variation| descendant, garde les 20 plus anomaliques → limite les chunks
- Batch de 5 : chaque chunk = 5 KPIs → prompt court, reponse rapide et precise
- Cache SHA-256 : meme analyse demandee 2 fois = 0ms la 2eme (cache hit)
- Index IVFFlat : recherche vectorielle en $O(\log n)$ au lieu de $O(n)$
- EmbeddingService cache : vecteur deja calcule n'est jamais recalcule dans la meme session

15. Deploiement Docker

Architecture Docker Compose

```
services:
  postgres:
    image: pgvector/pgvector:pg15
    volumes: [postgres_data:/var/lib/postgresql/data]
    env: POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD

  backend:
    build: ./QHSEAnalytics
    depends_on: [postgres] (condition: service_healthy)
    env: DB_*, JWT_SECRET, GROQ_API_KEYS, GEMINI_API_KEY, MAIL_*
    expose: [8080] (pas expose publiquement, reseau interne Docker)

  frontend:
    build: ./frontend (Angular build + Nginx)
    ports: [80:80]
    depends_on: [backend]

volumes:
  postgres_data: (persistance des donnees entre redemarrages)
```

Role de Nginx en production

- Sert les fichiers statiques Angular (dist/frontend/browser/) sur port 80
- Proxyfie /api/* vers http://backend:8080 (reseau Docker interne)
- Gzip compression sur les assets JS/CSS (gain 60-80% de taille)
- Cache-Control headers immutable pour les chunks haches (performance navigateur)

Variables d'environnement (.env)

Toutes les valeurs sensibles sont externalisees dans .env (jamais en dur dans le code). Docker Compose charge .env automatiquement. Spring Boot lit les variables via \${VAR_NAME} dans application.properties. Exemple :
app.groq.api-keys=\${GROQ_API_KEYS}

Flyway en production

Au demarrage du backend Docker, Flyway s'execute automatiquement. Si la DB est vide → applique V1-V7 dans l'ordre. Si la DB existe deja → applique uniquement les nouvelles migrations. Le container backend attend que postgres soit healthy (healthcheck + depends_on condition).

16. Questions-Reponses Soutenance

Architecture et choix technologiques

Q : Pourquoi Spring Boot plutot que FastAPI (Python) pour un projet IA ?

R : Le c
securite,
sont app
mais on a

Q : Pourquoi PostgreSQL et pas une base vectorielle dediee (Pinecone, Weaviate) ?

R : pg
vecteurs,
relationne
Pinecone

Q : Pourquoi Groq comme LLM primaire et pas OpenAI ?

R : Gro
OpenAI
developp

Pipeline d'import

Q : Comment gerez-vous les Excel avec des structures differentes ?

R :
KpiMatch
catalogue
Les temp

Q : Que se passe-t-il si valeurN1 est zero (division par zero) ?

R : Cor
(non calo

LLM et prompt engineering

Q : Que se passe-t-il si le LLM retourne un JSON malforme ?

R : S
sous-cha
invalide,
L'utilisati

Q : Comment garantisiez-vous la qualite des analyses IA ?

R : (1) F
sur les

RAG

Q : En quoi le RAG ameliore-t-il concretement les resultats ?

R : Sa
excellent
prioritaire
depasser

Q : Quelle est la difference entre similarite cosinus et distance euclidienne ?

R : Dis
mesure l'
direction

Securite

Q : Pourquoi JWT en cookie HttpOnly plutot qu'en localStorage ?

R : loc
token). C
restante

Q : Comment gerez-vous la revocation des tokens ?

R : Les
sont stoc
revoqu. L
vs revoca

Flow utilisateur complet

Q : Decrivez le flux complet d'utilisation de la plateforme de A a Z.

R : 1) I
active →
8) Uploa
Classifica
Declench
dans pro
/analyse-

Mots-cles a maitriser

LLM / Groq	Large Language Model. Groq = inference rapide via LPU. Parametres cles: temperature, max_tokens, response_format:json_object, rotation de cles API.
RAG	Retrieval-Augmented Generation. Enrichit le prompt LLM avec des donnees extraites d'une base de connaissances par recherche semantique vectorielle.
pgvector	Extension PostgreSQL pour stocker et interroger des vecteurs flottants. Operateur <=> = distance cosinus. Index IVFFlat pour ANN rapide.
Embedding	Representation numerique d'un texte en vecteur dense (768D ici via Gemini text-embedding-2). Textes semantiquement proches = vecteurs proches dans l'espace.
Similarite cosinus	Mesure l'angle entre 2 vecteurs. 1.0 = identiques, 0.0 = orthogonaux. Independante de la longueur. Formule: $(A.B)/(A B)$
Fallback chain	Groq (primaire) → Gemini (fallback). ProviderCooldownManager gere les cooldowns apres 429. Cache SHA-256 evite les appels redondants.
Flyway	Migration DB versionnee (V1→V7). Immuable : ne jamais modifier un script existant, toujours creer un Vn+1.
JWT HttpOnly	Token d'authentification en cookie inaccessible JS. Elimine XSS. Access 30min + Refresh 1h (7j remember-me) stocke en base.
Standalone Angular	Composants sans NgModule. Imports declares par composant. Tree-shaking optimal. Signals pour la reactivite.
ClassificationEngine	FAIBLE/MODERE/CRITIQUE selon direction du KPI (LOWER_IS_BETTER infere par mots-cles) et amplitude de variation.